

CSE 4345: Big Data Analytics

Week 8, Part 1 — Data Integration at Scale: Pipelines, Data Lakes & Modern Architectures

ROKAN UDDIN FARUQUI

Professor

Department of Computer Science & Engineering

University of Chittagong, Chittagong

CSE, PUC, Spring 2026

Course & Lecture Information

Lecture Identity

Course: CSE 4345
Week / Part: 8 of 11 | Part 1 of 2
CLO: CLO3, CLO4
PLO: PLO(e), PLO(f)

CLO3 & CLO4

CLO3: *Explain big data techniques and tools to store, manage, and analyse large datasets*

CLO4: *Solve big data problems and build a project applying big data tools and frameworks*

Course Progression

Week 7: Hive, HiveQL, SQL-on-Hadoop
Week 8: Pipelines, Kafka, Lambda/Kappa
Week 9: Data visualisation and dashboards

Part 1 Covers

- From ETL to pipelines: the shift in integration thinking
- Data integration tool landscape (Sqoop, Flume, Kafka, NiFi)
- Apache Sqoop: bulk RDBMS ↔ HDFS transfer
- Apache Flume: log ingestion pipelines
- Apache Kafka: append-only logs, topics, partitions, offsets
- Kafka producers, consumers, consumer groups, and retention
- Lambda architecture (Marz): Batch + Speed + Serving layers
- Kappa architecture (Kreps): stream-only, replay-based reprocessing
- Lambda vs. Kappa: trade-offs and when to use each
- Data lake architecture and the risk of the data swamp

Lecture Agenda

- 1 From ETL to Pipelines: A Shift in Architecture
- 2 Batch Integration: Sqoop and Flume
- 3 Apache Kafka: The Append-Only Event Log
- 4 Lambda Architecture
- 5 Kappa Architecture
- 6 Data Lakes and the Data Swamp Risk
- 7 Student Assessment Pool

From ETL to Pipelines: A Shift in Architecture

Why Pipelines Replace Point-to-Point ETL

The Old Model: Point-to-Point ETL

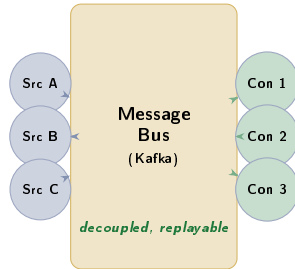
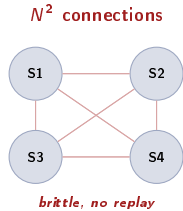
Traditional enterprises connected each pair of systems with a direct ETL job. With N systems, this creates $O(N^2)$ point-to-point connections. At 50 systems: potentially 2,450 separate ETL jobs to maintain.

- Brittle: every schema change in one system breaks its ETL jobs
- No replay: if a downstream consumer was offline, data is lost
- No real time: all jobs are nightly batch; latency ≥ 24 hours
- No decoupling: source and consumer are tightly bound

The New Model: Event-Driven Pipeline

Replace N^2 direct connections with a **central message bus**. Each source publishes events; each consumer subscribes. Adding a new consumer requires **zero changes** to any source system.

Why Pipelines Replace Point-to-Point ETL



Data Integration Tool Landscape

Tool	Direction	Type	Primary Use Case	Throughput
Apache Sqoop	RDBMS ↔ HDFS	Batch	Nightly bulk transfer of Oracle/MySQL tables to Hadoop	GB – TB/hour
Apache Flume	Logs → HDFS	Batch / micro-batch	Web server log ingestion; aggregating multi-source logs	MB – GB/hour
Apache Kafka	Any → Any	Real-time stream	Centralised event bus; fraud events, clickstream, IoT	GB – TB/hour
Apache NiFi	Any → Any	Visual pipeline	No-code data routing; IoT edge to cloud; file transfer	MB – GB/hour
Spark Streaming	Kafka → HDFS/DB	Stream processing	Stateful aggregation on streams; enrichment; joins	GB/hour

Batch Integration: Sqoop and Flume

Apache Sqoop: Bulk RDBMS ↔ HDFS Transfer

What Sqoop Does

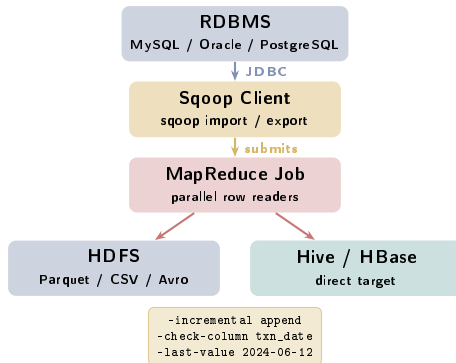
Sqoop (**SQL-to-Hadoop**) is a command-line tool that transfers data in bulk between a **relational database** (MySQL, Oracle, PostgreSQL, SQL Server) and **HDFS** (or Hive, HBase).

- **Import:** copies a table or query result from RDBMS → HDFS as CSV, Avro, Parquet, or Sequence File
- **Export:** copies data from HDFS → RDBMS (e.g., after Spark processing, push results back to MySQL)
- Internally launches a **MapReduce job** where each Map task reads a subset of rows (partitioned by primary key range) in parallel
- Supports **incremental import:** only rows newer than the last run (using a `-last-value` checkpoint on a timestamp or auto-increment column)

bKash Use Case

Nightly at 01:00, Sqoop imports the day's bKash wallet transactions (10 M rows) from the production Oracle OLTP database into HDFS as Parquet, partitioned by `txn_date`. Hive analytics then run on HDFS without touching the live database.

Apache Sqoop: Bulk RDBMS ↔ HDFS Transfer



Apache Flume: Log Ingestion Pipelines

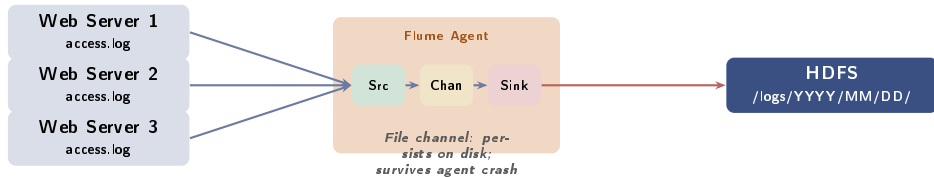
Flume Architecture: Source → Channel → Sink

Flume moves **semi-structured log data** (web server logs, application logs, IoT sensor readings) into HDFS for archival and batch analysis. Every Flume agent has three components:

- **Source**: reads incoming data. Types: Avro source (from another agent), Syslog source, Exec source (tail a log file), HTTP source
- **Channel**: buffers events between Source and Sink. Memory channel (fast, no persistence) or **File channel** (persistent, survives agent restart)
- **Sink**: writes data to the destination. HDFS sink (writes Avro/text files), Kafka sink (forward to Kafka), HBase sink

Flume supports **fan-in** (many sources → one agent) and **fan-out** (one source → many sinks) topologies, enabling hierarchical log aggregation across many servers.

Apache Flume: Log Ingestion Pipelines



Flume vs. Kafka

Flume is optimised for **log collection to HDFS**. Kafka is a **general-purpose event bus** with stronger durability and replay semantics. In modern stacks, Flume is often replaced by Kafka producers writing directly from applications.

Apache Kafka: The Append-Only Event Log

Apache Kafka: The Core Insight

Origin: LinkedIn, 2011

LinkedIn needed to move **activity events** (profile views, connection requests, job clicks, feed impressions) from dozens of producer services to dozens of consumer services (recommendation engines, analytics, monitoring). Point-to-point messaging created an N^2 web of dependencies.

Jay Kreps, Neha Narkhede, and Jun Rao designed Kafka around one insight:

The Founding Insight

*"A log is the most primitive form of storage. If you model your data infrastructure as a distributed, replicated, append-only log, you get **durability**, **replayability**, and **decoupling** for free."*

— Kreps, Narkhede & Rao (2011)

Why "append-only" matters

A traditional queue deletes a message after it is consumed. Kafka **retains all messages** for a configurable period (days, weeks, or forever). Late-joining consumers can read from offset 0 and replay the entire history. This enables new analytics pipelines to be built without re-ingesting from source.

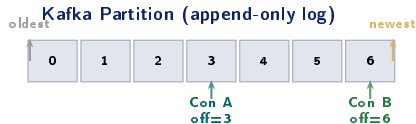
Apache Kafka: The Core Insight

Traditional Queue



consumed = deleted

late consumer misses data



Each consumer tracks its own offset

Messages retained for days/weeks

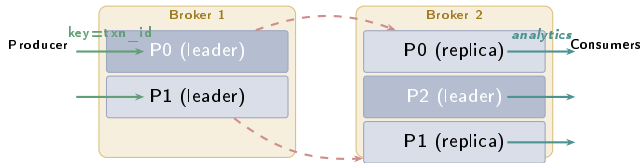
Kafka Architecture: Topics, Partitions, Brokers

Key Components

- **Topic:** a named, durable, append-only log for a category of events (e.g., `bkash-transactions`, `pathao-gps-pings`)
- **Partition:** each topic is split into P ordered partitions for parallelism. All messages with the same **key** go to the same partition (preserving key-level ordering)
- **Offset:** integer index of a message within a partition. Each consumer independently tracks its own read offset
- **Broker:** a Kafka server that stores partitions on disk. A cluster has B brokers; each partition has one **leader** and $R - 1$ **follower replicas** (default $R = 3$)
- **Producer:** publishes messages to a topic; selects partition via key hash or round-robin
- **Consumer Group:** a set of consumers that divide a topic's partitions between them for parallel consumption. Each partition is assigned to exactly one consumer in a group at any time
- **ZooKeeper / KRaft:** manages broker metadata and leader election (ZooKeeper in Kafka < 3.0 ; KRaft in Kafka ≥ 3.0)

Kafka Architecture: Topics, Partitions, Brokers

Topic: bkash-txns (3 partitions, RF=2)



How Kafka Achieves High Throughput

- **Sequential disk writes:** Kafka never updates messages in place. Append-only writes are sequential — modern HDDs achieve 600 MB/s sequential vs. <1 MB/s random. Kafka exploits this fully.
- **Zero-copy transfer:** Kafka uses the OS `sendfile()` system call to transfer data from disk to network socket without copying through user-space memory. Eliminates one full memory copy per message.
- **Batching:** producers and consumers batch multiple messages in one network round-trip. Configurable via `batch.size` and `linger.ms`.
- **Partition parallelism:** P partitions enable P producers and up to P consumers in a group to operate simultaneously — throughput scales linearly with partition count.

Kafka: Throughput, Retention, and Replay

Kafka at bKash and Pathao

bKash uses Kafka topics `wallet-credit` and `wallet-debit` to stream every transaction event to three consumers simultaneously:

- ❶ Fraud detection (Spark Streaming, <200 ms latency)
- ❷ Balance update (RDBMS writer)
- ❸ Audit log writer (HDFS, daily reconciliation)

Pathao publishes GPS pings every 3 seconds from active drivers to a Kafka topic `driver-location` (10 partitions, keyed by `driver_id`), consumed by the routing engine and the dispatch service independently.

Retention and Replay

Kafka retains all messages for a configured period (e.g., 7 days) or up to a storage quota. A new consumer can start at offset 0 and replay the entire week of events — enabling historical backfill without re-ingesting from the source database.

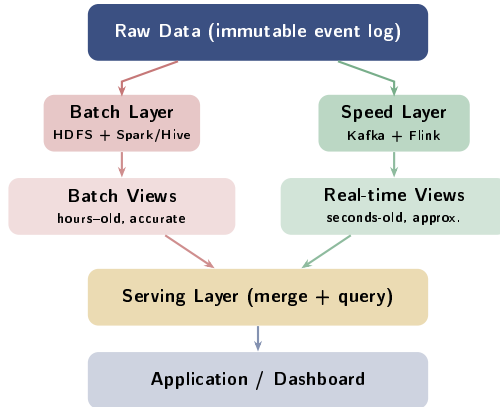
Lambda Architecture

Design Goal

Serve **both** accurate historical batch views and **low-latency** real-time views from the same data, with fault tolerance and the ability to recompute from scratch. **Three layers:**

- 1 **Batch Layer:** stores the complete, immutable master dataset (HDFS); periodically recomputes **batch views** (e.g., Spark/Hive jobs running every hour or nightly). Output is accurate but **high-latency** (1 hour to 24 hours stale).
- 2 **Speed Layer:** processes new data arriving since the last batch job using a stream processor (Kafka + Spark Streaming / Flink). Produces **real-time views** covering the most recent window. Output is **approximate but low-latency** (<seconds).
- 3 **Serving Layer:** merges batch views and real-time views to answer queries. A query result = *batch view (up to N hours ago) UNION real-time view (last N hours)*.

Lambda Architecture (Nathan Marz, 2011)



Lambda Architecture: Strengths and Weaknesses

Strengths of Lambda

- **Accuracy:** the batch layer always recomputes from raw data, so historical views are correct regardless of bugs or schema changes in earlier runs
- **Low latency:** the speed layer delivers results within seconds for the most recent data window
- **Fault tolerance:** the immutable master dataset on HDFS is the single source of truth; any derived view can be recomputed from it
- **Separation of concerns:** batch and speed components can be built, deployed, and scaled independently

The Critical Weakness: Dual Code Paths

Lambda requires implementing **the same business logic twice**:

- Once in the batch layer (Spark/Hive jobs)
- Once in the speed layer (Spark Streaming)

When business logic changes, **both implementations must be updated simultaneously and kept in sync**. Teams report that 30–40% of engineering effort goes into keeping the two layers consistent — a significant maintenance burden. **Additional complexity:**

- The serving layer must merge two query results at runtime
- Monitoring and debugging pipelines
- Two sets of infrastructure (Hadoop cluster + streaming cluster)

Kappa Architecture

The Simplifying Insight

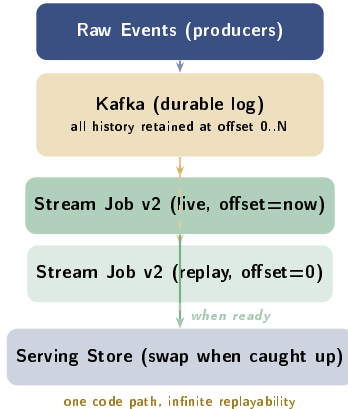
Jay Kreps (co-creator of Kafka) argued that the batch layer in Lambda is unnecessary if your stream processing system is **powerful enough to reprocess historical data** at the same speed as it processes live data. **Kappa's two components:**

- 1 **Log storage** (Kafka with long retention or Apache Iceberg on S3): stores the complete immutable event history. Replacing the HDFS batch layer.
- 2 **Stream processing engine** (Flink, Spark Structured Streaming): the **single code path** that handles both live events and historical replay. To reprocess history, start a new consumer at offset 0 and let it catch up.

Reprocessing workflow:

- Deploy a new version of the stream processing job
- Start it from Kafka offset 0 (beginning of history)
- It reads and processes all historical events, then seamlessly catches up to live events
- When caught up, swap it into production and retire the old version

Kappa Architecture (Jay Kreps, 2014)



Lambda vs. Kappa: Full Comparison

Property		Lambda Architecture	Kappa Architecture
Code paths		Two (batch + stream) — must stay in sync	One (stream only)
Historical reprocessing		Re-run batch job on HDFS	Replay Kafka log from offset 0
Latency		Low (speed layer) + High (batch layer)	Low throughout
Complexity		High (two systems, merging layer)	Lower (single pipeline)
Batch throughput		Very high (Spark/Hive on full dataset)	Limited by stream engine throughput
Long-term storage		HDFS (immutable master dataset)	Kafka (bounded) + external store (S3/Iceberg)
When to use Lambda		Business logic too complex for stream-only; batch ML training; regulatory batch reports	—
When to use Kappa		Stream engine is fast enough to reprocess; single code path is worth the stream-storage cost	—
Kappa when	NOT suitable	(1) Reprocessing TB+ history is too slow for the stream engine; (2) Batch SQL analytics (Hive, Spark batch) are preferred over stream SQL; (3) Regulatory requirement mandates a separate batch reconciliation system	

Data Lakes and the Data Swamp Risk

What Is a Data Lake?

A data lake is a centralised repository that stores **raw data at any scale, in any format** (structured, semi-structured, unstructured) without requiring upfront schema definition. Data is ingested cheaply and quickly; transformation happens on demand. **Four-layer architecture:**

- ❶ **Ingestion Layer:** Sqoop, Flume, Kafka, NiFi, REST APIs — pulls data from all sources and writes raw files to storage
- ❷ **Storage Layer:** HDFS or S3 / Azure ADLS — object storage organised in zones (raw → processed → curated)
- ❸ **Processing Layer:** Spark, Hive, Presto — transforms and queries data on demand
- ❹ **Catalogue / Governance Layer:** Apache Atlas, AWS Glue Data Catalog, Hive Metastore — tracks schemas, lineage, ownership, and access policies

The Data Swamp Problem

A data lake without governance rapidly becomes a **data swamp**: raw files accumulate without metadata, owners, or quality guarantees. Symptoms:

- Analysts cannot find relevant datasets (no catalogue)
- No one knows which files are current vs. stale
- Multiple copies of the same data with different transformations exist with no version control
- PII (personal data) is mixed with non-sensitive data, violating privacy regulations

Prevention: enforce a data catalogue at ingest time; assign owners to every dataset; implement access control; run automated quality checks; use a table format (Delta Lake, Apache Iceberg) to add ACID guarantees to the lake.

Student Assessment Pool

Instructions

Select the single best answer. Correct answers **bolded** for instructor reference.

- Q1.** Which tool is specifically designed to transfer data in bulk between an RDBMS (e.g., MySQL) and HDFS?
- ☐ (a) Apache Flume
 - ☐ (b) Apache Kafka
 - ☒ (c) **Apache Sqoop**
 - ☐ (d) Apache NiFi
- Q2.** Apache Kafka's data model is based on which fundamental storage abstraction?
- ☐ (a) B-tree indexes for fast point lookups
 - ☐ (b) Column-family storage (like HBase)
 - ☒ (c) **Append-only immutable distributed logs (partitioned topics)**
 - ☐ (d) Document storage with secondary indexes

Instructor Notes

Q1: Flume (a) is for logs to HDFS; Kafka (b) is for real-time streaming; NiFi (d) is a visual pipeline

Q3. In the Lambda architecture, which layer handles **real-time, low-latency** query views?

- ☐ (a) Batch Layer (b) Serving Layer (c) **Speed Layer** (d) Ingestion Layer

Q4. The Kappa architecture differs from the Lambda architecture primarily by:

- ☐ (a) Adding a third processing layer for machine learning
☐ (b) Using only relational databases for both batch and stream storage
☒ (c) **Eliminating the batch layer and using Kafka log replay for all reprocessing**
☐ (d) Replacing Hadoop with Spark in the batch layer while keeping the speed layer

Instructor Notes

Q3: The Serving Layer (b) is not a processing layer — it merges precomputed views and serves queries. The Batch Layer (a) is high-latency. The Speed Layer (c) is the real-time component. **Q4:** Option (d) is a common misconception — Kappa does not just replace Hadoop; it eliminates the entire batch layer and relies on a single stream processing code path with log replay for historical reprocessing.

Instructions

State True or False and provide a **one-sentence justification** for full marks.

Statement	Answer
1. Apache Sqoop is designed for real-time streaming ingestion from transactional systems.	False
2. Kafka retains messages for a configurable period, allowing consumers to replay events from any past offset.	True
3. The Lambda architecture requires maintaining two separate code paths implementing the same business logic.	True
4. Apache Flume is the preferred tool for collecting and aggregating server log data into HDFS.	True

Instructions

Answer each question in **100–150 words**. Include a diagram where indicated.

- D1.** Explain the **Lambda architecture** with a labelled diagram. What are its three layers? What is its primary engineering disadvantage compared to the Kappa architecture?
- D2.** Describe how Apache Kafka achieves **high throughput and fault tolerance**. Explain the roles of topics, partitions, offsets, brokers, and replication factor in the Kafka data model.
- D3.** What is a **data lake**? How does it differ from a data warehouse in terms of schema enforcement, storage format, and typical use cases? What is a “data swamp” and how is it prevented?

Instructions

Answer in **200–300 words** with step-by-step reasoning and tool justifications.

A1. Pipeline Design for a Bank:

Design a data integration pipeline for a Bangladeshi commercial bank that must:

- (a) Synchronise 10 million customer records nightly from Oracle to Hadoop for analytics
- (b) Detect fraudulent card-swipe events in real time with a decision within 200 ms
- (c) Power a risk management dashboard with data no older than 5 minutes

Specify which tool (Sqoop, Flume, Kafka, Spark Streaming) you would use at each stage. Justify each choice with reference to latency, throughput, and fault-tolerance requirements. Which architecture (Lambda or Kappa) fits best, and why?

A2. Lambda to Kappa Migration:

A team using Lambda architecture complains that maintaining two code paths (batch + stream) doubles development cost. Their CTO suggests migrating to Kappa. Analyse: (a) what must be true about the stream processing engine for Kappa to be viable; (b) what are the risks of migrating; (c) under what conditions is Kappa **not suitable** (give at least two concrete scenarios from the Bangladesh financial or healthcare sector)?

Textbook & Reading References

Primary Textbooks for Week 8

- **[TW]** Tom White. *Hadoop: The Definitive Guide*, O'Reilly, 4th ed., 2015. **Ch. 14, 15**
 - Ch. 14: Apache Flume — architecture, sources, channels, sinks, fan-in/fan-out
 - Ch. 15: Apache Sqoop — import/export, incremental, partitioning
- **[SA]** Acharya & Chellappan. *Big Data and Analytics*, Wiley, 2015. **Ch. 8**
 - Ch. 8: Data integration tools and architectures overview
- **[TE]** Erl, Khattak & Buhler. *Big Data Fundamentals*, Prentice Hall, 2016. **Ch. 10**
 - Ch. 10: Big Data integration patterns and enterprise architecture considerations

Graduate Reference and Research Article (covered in Week 8, Part 2)

- Kreps, J., Narkhede, N., & Rao, J. (2011). **Kafka: A Distributed Messaging System for Log Aggregation**. LinkedIn Engineering. [PRIMARY Research Article]
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly. Ch. 3 (storage engines), Ch. 10 (batch processing), Ch. 11 (stream processing). [Graduate-level systems reference at MIT, CMU, Stanford]

Questions & Discussion

Week 8, Part 1 | CSE 4345 Big Data Analytics

“I believe that a unified log is the central data structure that can solve the coherency problem in a distributed, heterogeneous data system — a structure so simple that we take it for granted, and so fundamental that we underestimate it.”

— Jay Kreps, *The Log*, LinkedIn Engineering, 2013

Next Lecture: Week 8, Part 2

Research Article: Kreps, Narkhede & Rao (2011) — *“Kafka: A Distributed Messaging System for Log Aggregation”*

Case Study: **Netflix Keystone Real-time Stream Processing Platform** — 500 billion events per day, Kafka + Flink + S3 + Apache Iceberg

Reading before Part 2: [TW] Ch.14–15 · Kreps et al. (2011) Kafka paper (see references)